

RippleLab Day 07 Progress Report

Day: 07 of 30

Date: 2026-08-14

Status: Complete

Branch: codex/day-07-repo-rate

Implementation commit: 735e4f0

Deployed commit: 735e4f0

Production version: 3

Quality gate: Full local suite passed

1. Objective

Deliver RippleLab's first complete personal economic simulation. Day 7 connects a signed-in user's saved financial profile to an editable repo-rate scenario, sends the typed request through a protected Next.js boundary to FastAPI, runs versioned deterministic loan and deposit formulas, and returns an inspectable personal impact result. No language model calculates money.

2. Acceptance Criteria

- [x] Require authentication before the scenario page or simulation API can be used.
- [x] Load the signed-in user's saved financial profile.
- [x] Convert the profile and explicit user assumptions into the canonical repo-rate request.
- [x] Run the request through FastAPI rather than calculating money in the browser.
- [x] Recalculate the floating-rate EMI with configurable loan-rate transmission.
- [x] Recalculate gross annual deposit income with configurable deposit-rate transmission.
- [x] Return annual loan, deposit and net impacts in integer paise.
- [x] Return assumptions, causal edges, citations, confidence, warnings and model version.
- [x] Show deterministic sensitivity bounds without presenting them as probability percentiles.
- [x] Produce a seeded homeowner-and-saver result that is exactly reproducible.
- [x] Let the user edit transmission assumptions and rerun without changing the saved profile.
- [x] Show Day 7 progress and the first personal result on the deployed progress site.

3. Delivered User Journey

The signed-in user saves or opens a financial profile, selects the repo-rate simulator and reviews prefilled assumptions. The browser sends the scenario to RippleLab's authenticated server route. That route forwards the canonical request to the economic engine using the server-only `RIPPLELAB_ENGINE_URL`. The engine calculates the loan and deposit paths, then returns a typed result for presentation.

Flow: saved profile and editable assumptions -> protected scenario page -> authenticated server route -> FastAPI endpoint -> versioned Decimal calculations -> typed impact, bounds, confidence and causal evidence.

The result screen separates the selected-assumption calculation from model uncertainty. It shows the modeled EMI before and after the reset, the annual loan benefit, the annual deposit cost, the annual net impact, lower and upper sensitivity bounds, confidence, assumptions and warnings.

4. Deterministic Architecture

Layer	Responsibility	Money calculation
Profile store	User-owned balances, income, loans and rates	None
React client	Edit scenario assumptions and render the typed result	None
Next.js server route	Authenticate and forward the canonical request	None
FastAPI orchestrator	Validate the scenario and compose the result	Orchestration only
Calculation primitives	EMI reset, rate transmission and deposit interest	Yes, deterministic
LLM boundary	Future parsing and plain-language explanation	Never

The public contracts remain the source of truth between TypeScript and Python. Money is transported as integer paise and rates as basis points. Python `Decimal` arithmetic performs intermediate calculations and final monetary values are rounded once to the nearest paise with half-up rounding.

5. Scenario Assumptions

Assumption	Seeded value	Unit	Why it is explicit
Repo-rate shock	-100	basis points	User-selected policy scenario
Loan pass-through	70%	ratio	Actual lender transmission is not guaranteed
Deposit pass-through	50%	ratio	Deposit repricing differs by bank and maturity
Remaining loan term	180	months	Current profile does not yet store the contractual term
Current deposit rate	7.25%	annual rate	Current profile stores the balance but not contracted rate
Horizon	12	months	Annualizes the first cash-flow view

The loan and deposit transmission ratios are editable. They are assumptions, not RBI promises or bank quotes. The causal edges cite the Reserve Bank of India's monetary-policy framework for policy-rate and transmission context; personal product terms remain supplied by the profile or user.

6. Formula Path

6.1 Floating home loan

The engine calls `loan.floating_rate_reset.v1` with the outstanding principal, current annual loan rate, remaining term, policy shock and loan pass-through ratio.

```

effective_loan_change_bps = repo_shock_bps * loan_pass_through
reset_loan_rate = current_loan_rate + effective_loan_change_bps
annual_loan_impact = -(reset_EMI - current_EMI) * 12

```

The remaining term is held constant. A lower EMI is a positive annual household cash-flow effect.

6.2 Fixed deposits

The engine calls `rates.linear_pass_through.v1`, rounds the modeled deposit rate to the nearest basis point and applies `deposit.simple_interest.v1` at the current and reset rates.

```

effective_deposit_change_bps = repo_shock_bps * deposit_pass_through
reset_deposit_rate = current_deposit_rate + effective_deposit_change_bps
annual_deposit_impact = reset_gross_interest - current_gross_interest

```

6.3 Household total

```

annual_net_impact = annual_loan_impact + annual_deposit_impact

```

7. Seeded Reproducible Result

Input	Value
Outstanding floating home loan	INR 65,00,000
Current loan rate	8.50%
Remaining term	180 months
Fixed deposits	INR 8,00,000
Current deposit rate	7.25%
Repo-rate change	-100 basis points
Loan pass-through	70%
Deposit pass-through	50%

Output	Deterministic result
Modeled EMI before reset	INR 64,008.07 per month
Modeled EMI after reset	INR 61,369.23 per month
Annual loan cash-flow benefit	INR 31,666.08
Annual deposit-income cost	INR 4,000.00
Annual net benefit	INR 27,666.08
Overall confidence	Medium, 75/100
Lower sensitivity result	INR 17,088.16 per year
Upper sensitivity result	INR 38,187.60 per year

The golden request and result are generated from this engine behavior. Repeated calls with the same inputs produce the same financial outputs. In the browser regression test, changing only loan pass-through from 70% to 50% changes annual net benefit from INR 27,666.08 to INR 18,688.16 while the saved profile remains unchanged.

8. Sensitivity, Confidence and Evidence

Day 7 does not claim a probability distribution and does not run Monte Carlo simulation. The lower and upper values are deterministic sensitivity checks. The engine moves both transmission assumptions down and up by 20 percentage points, clamps them to the valid 0%-100% range and evaluates the four corner combinations.

The cross-scenario contract retains fields named p10, p50 and p90, but the response method is `deterministic_bounds`. The interface therefore labels them lower, selected and upper rather than statistical percentiles.

Confidence is medium because the mathematical mapping is stable but actual bank transmission, reset dates, benchmark spreads and deposit renewal timing are unknown. Each causal edge contains a mechanism, lag range, assumption references, citation references and its own confidence assessment.

9. API and Error Boundaries

`POST /v1/simulations/repo-rate` accepts the canonical `SimulationRequest` and returns `SimulationResult` model version 1.0.0. Pydantic validation failures return HTTP 422 with `INVALID_SIMULATION_REQUEST`, a readable summary and field-level issues. Missing or invalid named assumptions return HTTP 422 with `INVALID_REPO_RATE_ASSUMPTION` and a direct corrective message.

The browser does not receive the economic-engine URL. The authenticated Next.js route reads it server-side, forwards the request and turns invalid JSON, engine connectivity failures and upstream errors into short product-facing messages.

10. Test and Verification Evidence

Check	Result
Economic-engine API and calculation suite	39 passed
Golden contract equality	Passed
Repeatability and editable-assumption tests	Passed
Python lint and formatting	Passed
TypeScript contracts, generated routes and type check	Passed
Next.js production build	Passed
Browser suite, desktop and mobile Chromium	22 passed; 4 intentional device skips
Result-state accessibility check	No serious or critical violations
Profile ownership, RLS, money and consent assertions	Passed
Repository secret scan	Passed
Sites Vinext production bundle	Passed
Sites production deployment	Version 3 succeeded

The Python suite reports one dependency deprecation warning from FastAPI's Starlette test client regarding `ht tpx`. It does not affect the 39 passing tests and remains dependency-maintenance work rather than a Day 7 functional failure.

11. Production Deployment

The Day 7 progress build is live at ripplelab-progress.nayaksiddhartha397.chatgpt.site. Sites version 3 was built, saved and deployed from exact pushed commit 735e4f0. Access remains owner-only during development.

The public product and progress surfaces are deployable. The authenticated repo-rate flow is complete and verified locally, but the hosted simulator is not yet operational because production Supabase credentials and a hosted FastAPI endpoint have not been configured. No placeholder credentials were committed. This boundary is intentionally reported instead of presenting the deployed progress page as a fully wired financial service.

12. Important Files

- `services/economic-engine/src/ripplelab_engine/repo_rate.py` and `main.py` - scenario orchestration, endpoint and structured errors.
- `services/economic-engine/tests/test_repo_rate.py` - golden, repeatability, assumption and endpoint tests.
- `packages/contracts/examples/repo-rate-request.v1.json` and `repo-rate-result.v1.json` - exact canonical fixtures.
- `apps/web/src/app/scenarios/repo-rate/page.tsx` and `apps/web/src/components/scenarios/repo-rate-simulator.tsx` - protected page, inputs and result.
- `apps/web/src/app/api/simulations/repo-rate/route.ts` - authenticated server-side engine boundary.
- `apps/web/tests/repo-rate.spec.ts` - end-to-end personal scenario and assumption update.
- `docs/REPO_RATE_VERTICAL_SLICE_V1.md` - scenario reference; `progress/page.tsx` and `public/og.png` - milestone sharing experience.

13. Decisions and Limitations

- Keep all monetary calculations in the deterministic Python engine; the browser owns inputs and presentation only.
- Keep the engine URL server-side and every incomplete profile field an explicit editable assumption.
- Distinguish the selected-assumption calculation from uncertainty.
- Use deterministic bounds now and defer probability language until a calibrated Monte Carlo model exists.
- Version formulas and scenario models independently for reproducibility.
- Preserve the remaining term when comparing EMIs; treat deposit income as gross simple interest before tax.
- Exclude fees, prepayments, benchmark floors, lender reset schedules, compounding, penalties and future policy changes.
- Present all output as educational simulation, not financial advice.

No code blocker remains for Day 7.

14. Day 8 Handoff

Day 8 should turn the result's causal nodes and edges into the first interactive causal graph. It should render the repo-rate, lending-rate, EMI, deposit-rate and deposit-income paths; let users inspect mechanisms, lag ranges, assumptions, citations and confidence; and keep the selected impact synchronized with the result cards. The graph must visualize the engine response rather than duplicate economic logic in the frontend.

Sign-off: Prepared by Codex | Evidence reviewed: Yes | Ready to continue: Yes